

University Course Registration System

Software Engineering — Group 06

1. Group Info

Group Number	Group 06
Case Study	University Course Registration System
Course	Software Engineering
Instructor	Samer Elkababji

Team Members

Lily Alalami	20230649
Nour Shalab	20230303
Abdullah Mashharawi	20221175
Rawan Abdo	20230946

2. Overview

System Purpose

This project is about building a web-based course registration system for a university. The idea is to give students a way to browse courses, sign up for them, drop or swap if needed, and check their grades and schedule — all in one place. The registrar handles things like opening and closing registration windows and managing how many students can join each course. Instructors can upload grades, see who's in their classes, and post announcements. The system also connects to external services for login, payment checks, and transcript generation.

Tools Used

- **Visual Studio Code** — main development environment
- **Draw.io** — used for C4 diagrams, use case diagram, sequence diagrams, and activity diagram
- **PlantUML** — used for the class diagram and DFD
- **GitHub** — version control and team collaboration
- **Pandoc** — converting the report to PDF

3. Diagrams

Diagram	Type	Description
C4 Level 1 — Context Diagram	C4 Model	Shows who interacts with the system and what external systems it connects to
C4 Level 2 — Container Diagram	C4 Model	Breaks the system into its main parts: web app, server, and database
Use Case Diagram	UML	Shows what each actor can do in the system
Use Case Descriptions	UML (Tabular)	A table going through each use case in detail
High-Level Sequence Diagram	UML	Shows how the main actors communicate with each other
Detailed Sequence Diagram	UML	Same as above but includes error cases and alternative flows
Class Diagram	UML	Shows the structure of the system — classes, attributes, and relationships
Activity Diagram with Swimlanes	UML	Shows step-by-step workflows across different actors
DFD Level 0	DFD	Context-level view of data entering and leaving the system
DFD Level 1	DFD	Breaks the system into five core processes and their data stores
DFD Level 2 (Processes 1–5)	DFD	Detailed breakdown of each individual process

4. Repository Structure

/docs/report.md	Markdown source file for the report
/docs/report.docx	Report written in Word format
/docs/report.pdf	Final PDF version of the report
/docs/images/	All exported diagram images
/uml/*.puml	PlantUML source files
/uml/*.drawio	Draw.io source files
README.md	Group info and project overview

5. Contributions

Member	Role & Contributions	Commits
Lily Alalami	Report writing, PPT, sequence diagram explanations, high-level and detailed sequence diagrams	45
Nour Shalab	C4 Level 1 & 2 diagrams, DFD diagram and description, C4 and DFD explanations	41
Abdullah Mashharawi	Class diagram, activity diagram with swimlanes, activity and class diagram explanations	42
Rawan Abdo	Use case diagram, use case description tables, generalization and aggregation on class diagram, related explanations	41

GitHub Repository: <https://github.com/abdullah-mashharawi/Group-6-Software-Engineering>

System Description

1.1 Introduction

The University Course Registration System is a web-based platform built to handle the course enrollment process for students, instructors, and the registrar's office. Instead of dealing with manual paperwork or disconnected systems, everything is managed in one place — from browsing courses to generating transcripts. The goal is to make the process faster, more consistent, and easier for everyone involved.

1.2 What the System Does

At its core, the system lets students manage their academic schedule. They can look through available courses, register for ones they want, drop courses they no longer need, or swap one course for another. The system checks a few things before letting a student register — do they meet the prerequisites, is there still space in the course, is the registration window open, and do they have any financial holds. All of these checks happen automatically before anything gets confirmed.

On the admin side, the registrar controls when registration opens and closes, and sets the maximum capacity for each course. Instructors can log in to see who's in their classes, upload grades, and post announcements for their students. When a student needs an official transcript, the system sends a request to an external transcript service that generates and delivers it.

1.3 Actors

There are four main actors in this system:

- **Student** — the main user. Can browse courses, register, drop, swap, view their schedule, and check their grades.
- **Instructor** — can manage their course roster, submit grades, and post announcements.
- **Registrar** — handles the administrative side. Sets registration timelines, manages course capacities, and oversees enrollment policies.
- **External Systems** — three outside services the system talks to: the Authentication System (handles login), the Finance System (checks if a student has any payment holds), and the Transcript System (generates official transcripts).

1.4 Core Features

- **Login and session management** — all users log in through the institutional authentication service. Sessions are checked before any protected action is allowed.
- **Course browsing** — students can view the full list of available courses, including details like section info, timings, and seat availability.
- **Registration** — students register for courses after the system checks prerequisites, capacity, financial status, and whether registration is open.
- **Drop and swap** — students can remove a course or replace it with another, as long as it's within the deadline. Swapping combines both a drop and a new registration.
- **Schedule and grade viewing** — students can check their current schedule and any grades that have been uploaded.
- **Grade submission** — instructors submit grades, which get validated and stored.
- **Roster management** — instructors can view and manage their enrolled student list.
- **Announcements** — instructors can post updates that all students in that course can see.
- **Transcript generation** — students can request an official transcript, which is generated by the external transcript service.

1.5 System Architecture

The system follows a standard three-tier web architecture. The front end is a web application that all users interact with through their browser. Behind that is an application server that handles all the logic — checking prerequisites, enforcing deadlines, validating payments, and so on. Then there's a relational database that stores everything: student records, courses, enrollments, grades, and more.

The application server is also the only part that talks to the external systems. So when a student logs in, the server contacts the authentication service. When they register, it checks with the finance system. When a transcript is needed, it sends the request to the transcript service. Keeping all of that in one place makes the system easier to maintain and update.

1.6 Data

The database holds all the core data: users (students, instructors, registrars), courses, sections, enrollments, prerequisites, grades, schedules, announcements, and transcripts. The application server enforces all rules before writing anything to the database, so data stays consistent.

1.7 External Integrations

The three external systems — Authentication, Finance, and Transcript — are connected through the application server using defined interfaces. This means if the university decides to change one of those services, only the server-side integration needs to be updated, not the whole system. It keeps things flexible without breaking what already works.

Part I: Context

C4 Level 1 — Context Diagram

The context diagram gives the highest-level view of the system — basically, who's involved and what outside services it talks to. There's no internal detail here, just the big picture of where the system sits within the university environment.

Actors

- **Student** — uses the system to browse courses, register, drop or swap, and view their schedule and grades.
- **Instructor** — logs in to manage their roster, submit grades, and post announcements.
- **Registrar's Office** — the admin side. Sets enrollment rules, manages timelines, and monitors registrations.

External Systems

- **Authentication System** — verifies who's logging in before granting access.
- **Finance System** — checks whether a student has any unpaid holds before allowing enrollment.
- **Transcript System** — receives academic data and produces official transcripts when requested.

The diagram makes it clear that the system sits in the middle — every interaction between a human actor and an external service goes through it. Nothing talks directly to the external systems except the core platform.

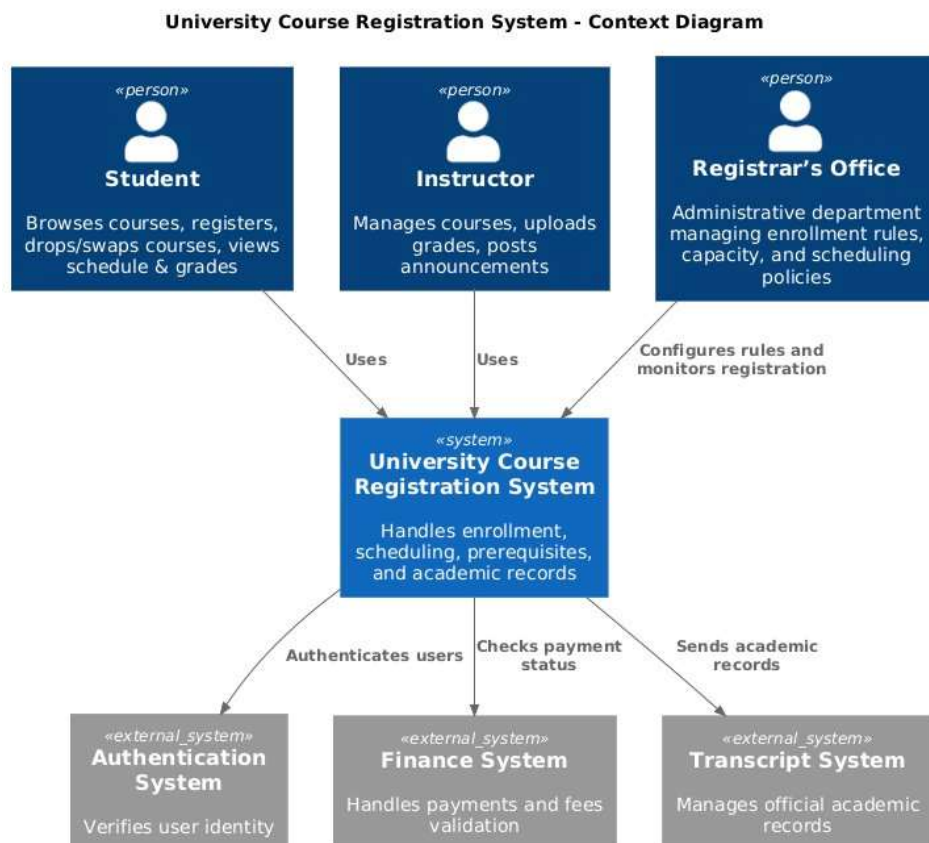


Figure 1: C4 Level 1 — Context Diagram

C4 Level 2 — Container Diagram

The container diagram goes one level deeper and breaks the system into its actual deployable components. This is where you can see how the different parts of the system are structured and how they communicate.

Containers

- **Web Application** — the front-end interface built with HTML, CSS, and JavaScript. This is what students, instructors, and the registrar see and interact with. It sends requests to the application server and displays the responses.
- **Application Server** — the core of the system. All business logic lives here — enrollment rules, prerequisite checks, scheduling, grading policies. It also handles all communication with external systems.
- **Relational Database** — stores everything: students, courses, sections, enrollments, prerequisites, grades, announcements, schedules.

External System Communication

- **Authentication System** — the server sends credentials here during login and session checks.
- **Finance System** — called before enrollment to confirm no financial holds exist.
- **Transcript System** — receives export requests and generates the official academic transcript.

The structure is a clean three-tier setup. The web app doesn't talk to the database directly, and it doesn't touch external systems. Everything goes through the application server, which acts as the single point of control.

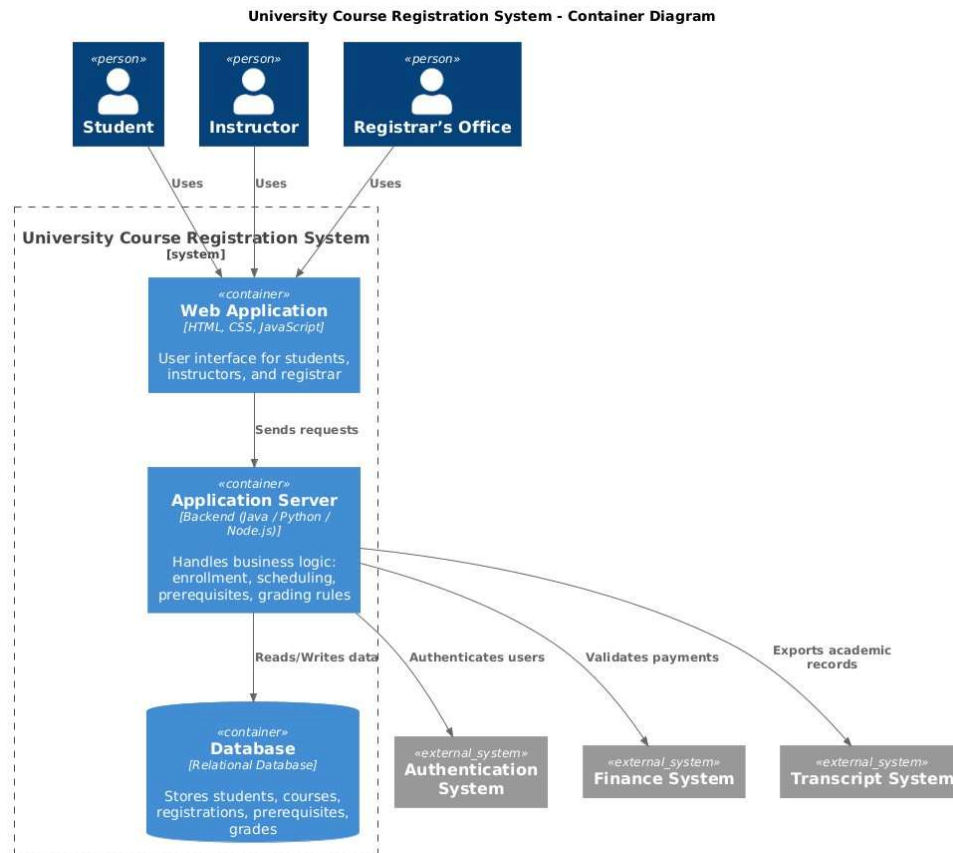


Figure 2: C4 Level 2 — Container Diagram

Part II: Interactions

Use Case Diagram

The use case diagram gives an overview of the system's functionality from the user's perspective. It maps out which actors are involved in which operations, and shows some of the relationships between use cases.

Actors

- **Student** — initiates all enrollment-related actions and academic queries.
- **Instructor** — manages course content, grades, and announcements.
- **Registrar** — handles administrative controls like timelines and capacities.
- **External Systems** — participates in login and can query grades.

Relationships Between Use Cases

- **Register for Course includes Verify Prerequisites and Check Capacity** — both are mandatory sub-steps that run automatically every time a student tries to register.
- **Swap Course includes both Drop Course and Register for Course** — it's essentially both operations combined into one.
- **View Schedule includes Browse Courses** — you can't build a schedule view without pulling course data first.
- **Post Announcements extends Upload Grades** — announcements can be triggered as a follow-up to a grade upload.

The diagram groups use cases by actor responsibility which makes it easy to see at a glance what each person in the system can actually do.



Figure 3: Use Case Diagram

Use Case Descriptions

The table below goes through each use case in more detail — who's involved, what data is needed, what triggers it, and what the system does in response.

Use Case	Actors	Description	Data	Stimulus	Response	Comments
Login	Student, Instructor, Registrar, External Systems	User logs into the system	Username, Password	User enters credentials	System validates and grants/denies access	Common authentication use case
Browse Courses	Student	View available courses	Course list	Student opens course catalog	System displays all available courses	
Register for Course	Student	Student enrolls in a course	Student ID, Course ID, Prerequisites, Capacity	Student selects a course	System runs prerequisite and capacity checks then enrolls the student	Includes: Verify Prerequisites, Check Capacity
Verify Prerequisites	System	Checks if student meets course requirements	Student record, course requirements	Triggered during registration	System validates eligibility	Included use case for registration
Check Capacity	System	Checks if course has available seats	Course ID, enrollment count, max capacity	Triggered during registration	System allows or rejects registration	Included use case for registration
Drop Course	Student	Student removes a course	Student ID, Course ID	Student selects drop option	System removes student from course	
Swap Course	Student	Replace one course with another	Student ID, Old Course ID, New Course ID	Student selects swap option	System drops old course and registers new one	Includes Drop Course + Register for Course
View Schedule	Student	Display student timetable	Enrollment data	Student requests schedule	System shows weekly schedule	
View Grades	Student, External Systems	View academic grades	Student ID, Grades	User requests grades	System displays grades	External systems may query grades
Manage Course Roster	Instructor	Manage enrolled students	Course ID, Student list	Instructor updates roster	System updates course enrollment list	
Upload Grades	Instructor	Enter student grades	Course ID, Grades	Instructor submits grades	System stores grades	Must validate data format
Post Announcements	Instructor	Publish course announcements	Announcement text, Course ID	Instructor posts update	System publishes announcement	
Manage Course Capacity	Registrar	Set maximum seats per course	Course ID, Capacity limit	Registrar updates capacity	System updates course limits	
Manage Registration Timeline	Registrar	Control registration period	Start date, End date	Registrar sets timeline	System opens/closes registration	

Table 1: Use Case Descriptions

High-Level Sequence Diagram

This diagram shows how the main actors communicate with each other during the system's key operations. It's meant to give a stakeholder-level view — the focus is on the overall flow rather than technical details or error handling.

- **Authentication:** The student sends a login request to the system. The system forwards this to the Institutional System for credential verification. Once cleared, the student gets a login success message.
- **Student Enrollment:** After logging in, the student initiates an enrollment request. The system checks with the Finance System to confirm there are no holds. If everything is clear, enrollment is confirmed.
- **Course Browsing:** The student requests the course list. The system returns all available courses.
- **Registration:** The student sends a registration request to the Registrar's Office. The registrar verifies the student's eligibility and tells the system to save the enrollment. Once that's done, the student gets a registration confirmed message.
- **Drop or Swap:** The student sends a drop or swap request. The registrar instructs the system to update the record. After the update, the student gets a confirmation.
- **Academic Schedule and Grades:** The student can request their schedule or grades at any time. The system retrieves and returns the relevant data.
- **Instructor Operations:** The instructor uploads grades, which the system saves. They can also request the course roster or post an announcement — both go through the system and get stored in the database.
- **Transcript:** The student requests a transcript. The system tells the Institutional System to generate it. Once it's ready, the transcript gets delivered back to the student.

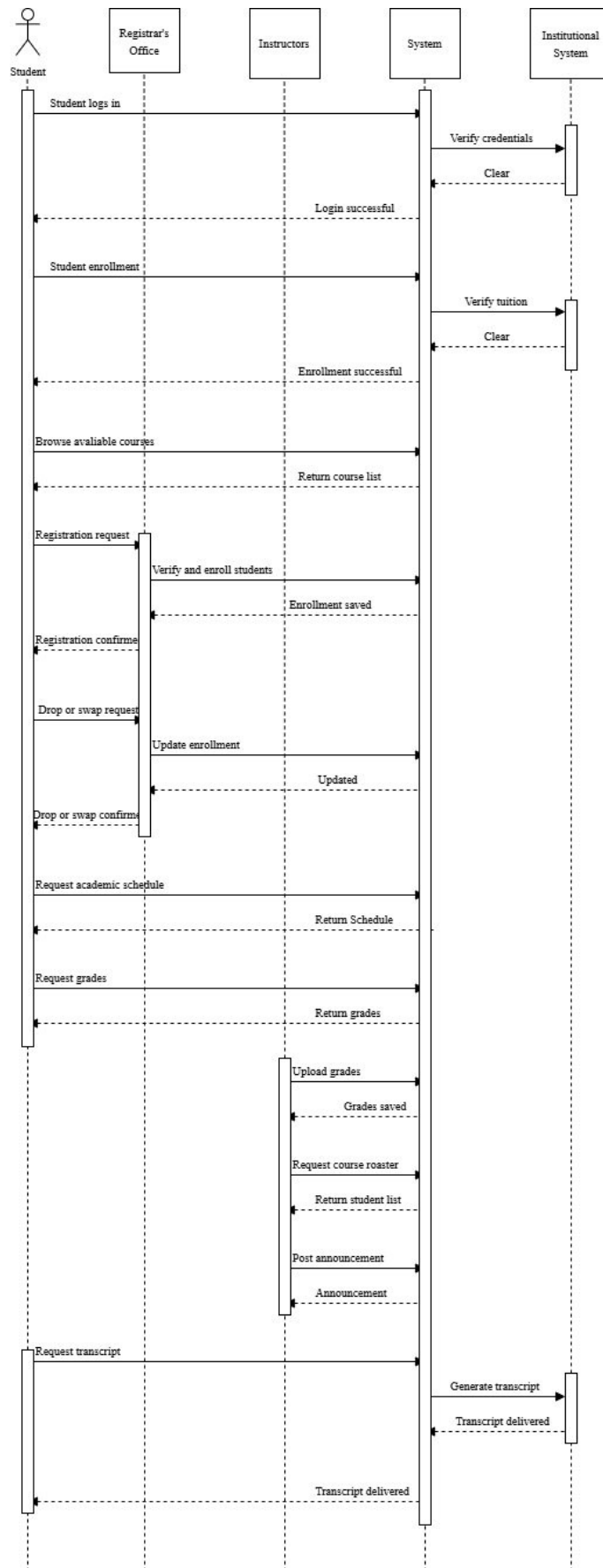


Figure 4: High-Level Sequence Diagram

Detailed Sequence Diagram

This version of the sequence diagram is aimed at developers. It covers the same flows as the high-level diagram but adds alternative paths, edge cases, and what happens when something goes wrong.

- **Login — What if credentials are wrong?** When the student logs in, the system checks credentials with the Institutional System. If they're valid, an authentication token is issued. If not, the system returns an invalid credentials error and access is denied.
- **Enrollment — What if there's a financial hold?** After login, the student tries to enroll. The system checks with the Finance System. If no hold exists, enrollment goes through. If a hold is detected, the system returns a hold error and the enrollment is blocked.
- **Registration — Validation Chain:** When the registration window is open, the system runs checks in sequence. First, prerequisites — if not met, the registration fails right there. Then capacity — if the course is full, it's rejected. Only if both pass does the enrollment get confirmed.
- **Drop and Swap — Deadline and Conditions:** For a swap within the deadline: the system checks prerequisites and capacity for the new course. If both are fine, the old course is dropped and the new one is added. If not, the swap is rejected. For a regular drop: the enrollment record is removed and the seat count goes back up. If the deadline has already passed, the system returns a period closed message and nothing changes.
- **Grades — What if they haven't been uploaded yet?** When a student requests their grades, the system checks if the instructor has submitted them. If yes, they're returned. If not, the system returns a grade pending response.

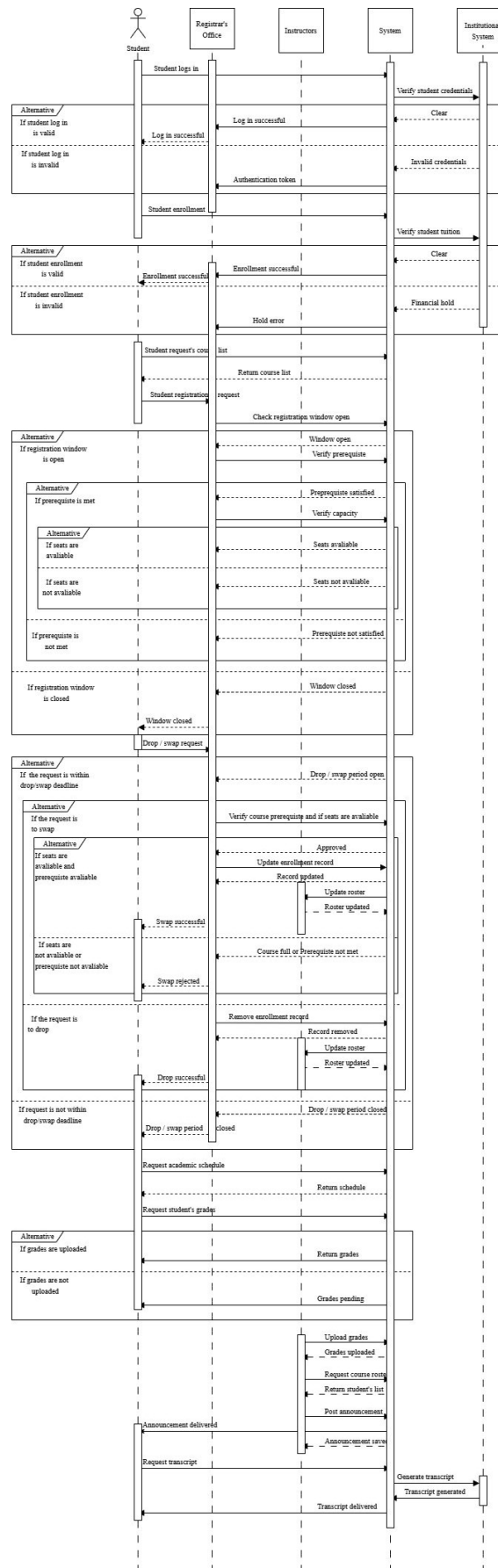


Figure 5: Detailed Sequence Diagram

Part III: Structure

Class Diagram

The class diagram shows the full structure of the system — all the classes, what they contain, and how they relate to each other. It uses five types of relationships: generalization, aggregation, composition, association, and dependency.

- **Abstract Classes and Inheritance:** The system uses abstract classes to avoid repeating the same code across similar entities. The User class is the parent of Student, Instructor, and Registrar — they all share things like a user ID, name, email, and the ability to log in and validate a session. The three external service classes (AuthenticationSystem, FinanceSystem, TranscriptSystem) all inherit from SystemComponent. Academic record types like Transcript, Grade, and Enrollment come from AcademicRecord. Course and Section both extend CourseOffering. Course and Section both extend CourseOffering.
- **Aggregation and Composition:** A course aggregates multiple Sections, meaning a course can have several scheduled offerings and they exist independently of each other. A Transcript aggregates multiple Grades — it's made up of individual grade entries. The only composition in the diagram is between Section and Announcement. An announcement can't exist on its own; it belongs to a specific section. If the section goes away, so does the announcement.
- **Associations and Dependencies:** The Enrollment class sits between Student and Section — it's the record that says a student is in a particular course offering. A Student is also linked to a Schedule that tracks their current timetable. The student class depends on both AuthenticationSystem and FinanceSystem, which reflects the fact that login and financial checks have to happen before enrollment. Course has a self-association to handle prerequisites — a course can require another course.

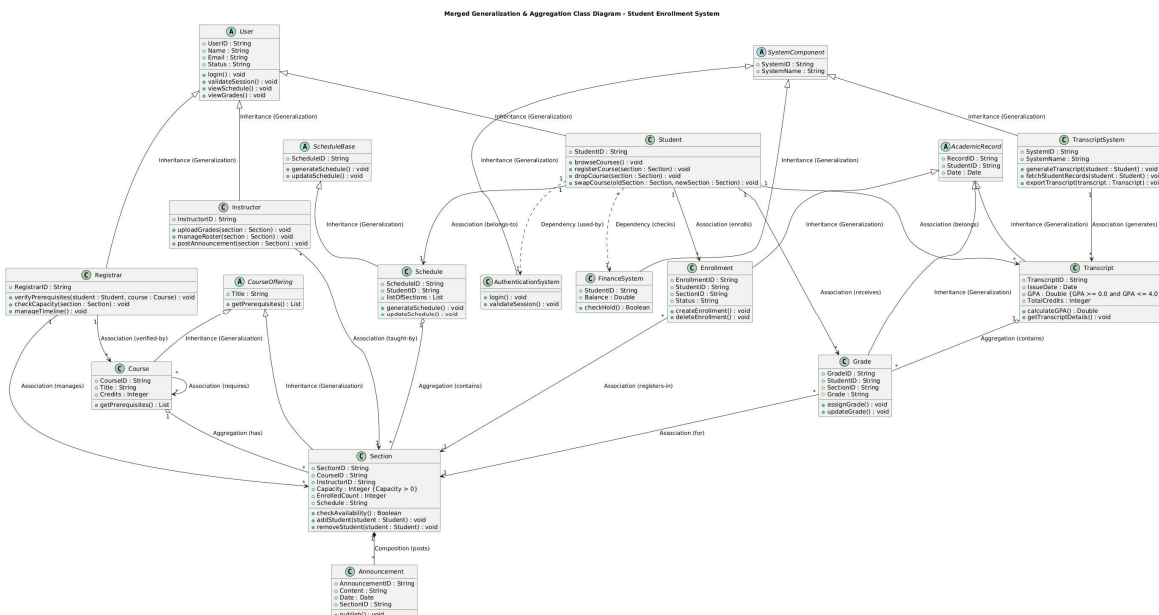


Figure 6: Class Diagram

Part IV: Behavior

Activity Diagram with Swimlanes

The activity diagram shows how the system actually behaves when users interact with it. It uses swimlanes to separate what each actor or system is responsible for, and decision nodes to show where the flow can branch.

- **Login and Entry:** Everything starts with the student logging in through the Authentication System. Once authenticated, a decision node lets them choose what to do next — register, drop, swap, or view academic info.
- **Registration:** For registration, the system first checks the session is still valid. If not, the student has to log in again. Then it checks with the Registrar whether the registration window is open. If closed, the process stops. If open, the system runs three checks in order: prerequisites (Registrar), seat capacity (database), and financial hold (Finance System). Fail any one and registration ends. Pass all three and the system uses a fork node to do three things at once — update the academic record, update the course roster, and adjust seat count. Once those all finish, the student gets a confirmation and their updated schedule.
- **Drop and Swap:** Dropping a course removes the student's enrollment record and frees up a seat. These two things happen in parallel. The student then sees their updated schedule. Swapping combines a drop and a registration — the old course is removed first, then the system tries to register the new one with all the same checks. If it works, everything is updated. If not, the student gets an error.
- **Viewing Academic Info:** If the student just wants to see their schedule or grades, the system retrieves the data from the database — no prerequisite or financial checks needed. The schedule and grade retrieval can happen at the same time, and the result is displayed to the student.
- **Instructor Side:** Instructors log in and can do three things: manage the roster, upload grades, or post an announcement. Uploading grades writes directly to the database. Posting an announcement stores it so enrolled students can see it.

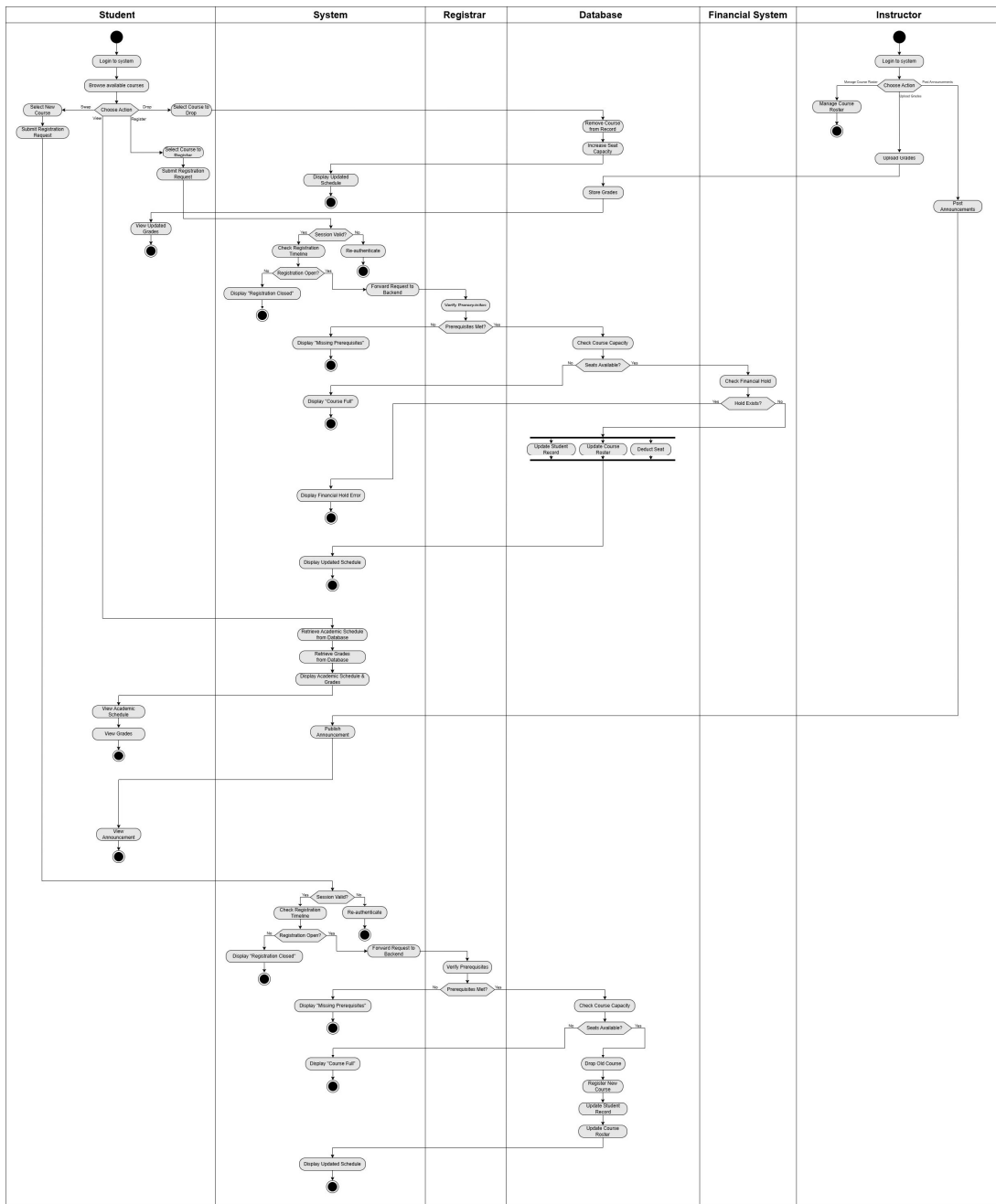


Figure 7: Activity Diagram with Swimlanes

Part V: Data Flow Diagrams

The Data Flow Diagram models how data moves through the University Course Registration System across five core processes. The DFD is broken into three levels of detail: Level 0 gives the big picture, Level 1 breaks the system into its main processes, and Level 2 drills into each process individually.

DFD Level 0 — Context Diagram

Level 0 shows the entire system as a single process and focuses on what data enters and leaves it. Three human actors interact with the system: the student sends course registration, drop, and swap requests and receives back confirmations, schedules, and grades. The instructor submits grades and course updates and gets back roster and enrollment statistics. The Registrar feeds in enrollment rules and capacity configurations and receives reports in return. Three external systems are also connected: the Authentication System handles credential verification, the Finance System processes payment checks, and the Transcript System receives academic records for official transcript generation.

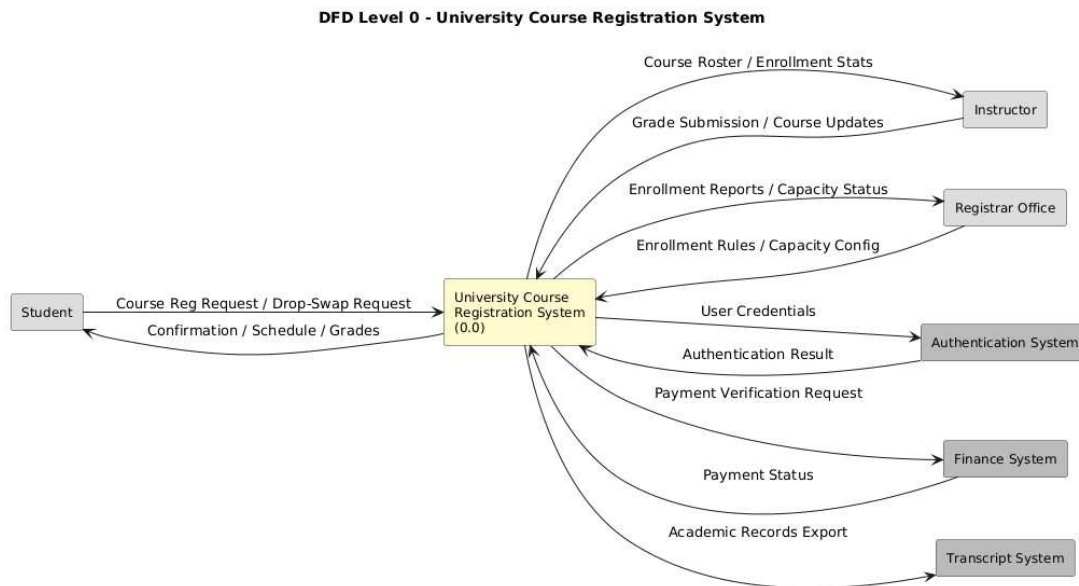


Figure 8: DFD Level 0 — Context Diagram

DFD Level 1 — System Processes

Level 1 expands the single system bubble into five distinct processes and introduces the data stores that support them.

- **Process 1.0 — Authenticate Users:** Handles login for all three actors. Credentials are verified against the User DB and through the external Authentication System. A session token is generated and stored on success, or access is denied on failure.
- **Process 2.0 — Manage Course Registration:** Handles all enrollment actions — registration, drop, and swap. It runs prerequisite checks, seat availability checks, and financial hold checks before confirming any enrollment. It also triggers payment processing through Process 4.0.
- **Process 3.0 — Manage Courses and Grades:** Covers the instructor side — updating course info, managing rosters, uploading materials, submitting grades, and posting announcements. The Registrar can also feed in capacity and scheduling approvals here.
- **Process 4.0 — Process Payments:** Handles fee calculation, payment verification with the Finance System, and storing transaction records. Updates the Enrollment DB with payment status once confirmed.
- **Process 5.0 — View Academic Records:** Handles student requests for transcripts and academic history. Pulls data from the User DB, Enrollment DB, and Grades DB, compiles it, and sends it to the Transcript System for export.

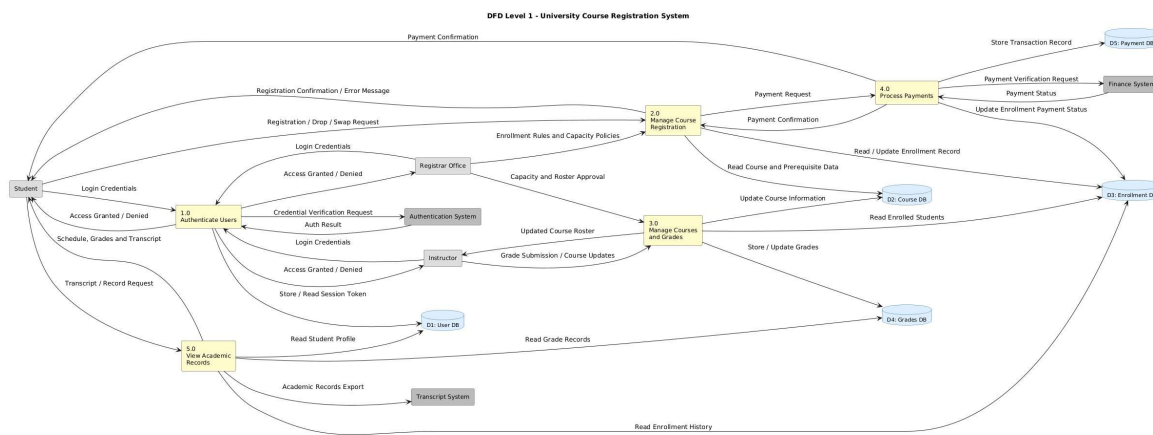


Figure 9: DFD Level 1 — System Processes

DFD Level 2 — Process Breakdowns

Process 1.0 — Authenticate Users

- **1.1 Capture Credentials:** Receives username and password from any actor.
- **1.2 Validate Against User DB:** Looks up the user record in the database to check if the user exists.
- **1.3 Verify via External Auth System:** Forwards credentials to the institutional Authentication System and receives the result.
- **1.4 Generate Session Token:** On successful verification, creates a session token and stores it in the User DB.
- **1.5 Return Access Result:** Sends back either access granted (with role and token) or access denied to the requesting actor.

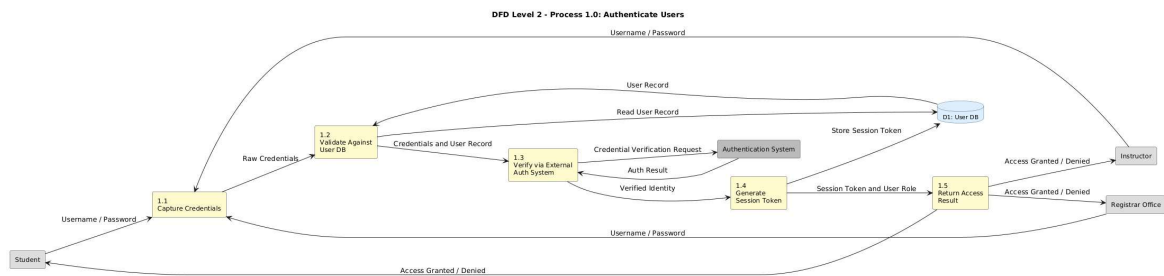


Figure 10: DFD Level 2 — Process 1.0: Authenticate Users

Process 2.0 — Manage Course Registration

- **2.1 Check Prerequisites:** Reads course prerequisite rules from the Course DB and verifies the student meets them. Fails immediately if not.
- **2.2 Check Seat Availability:** Reads current enrollment count and max capacity. Rejects if the course is full.
- **2.3 Check Financial Hold:** Contacts the Finance System to confirm no outstanding holds exist. Blocks registration if a hold is found.
- **2.4 Add / Drop / Swap Course:** Applies the actual enrollment change, enforcing the registration timeline set by the Registrar.
- **2.5 Update Enrollment Record:** Saves or updates the enrollment entry in the Enrollment DB.
- **2.6 Generate Payment Request:** Calculates fees, sends a payment verification request to Finance, and updates the enrollment payment status once confirmed.

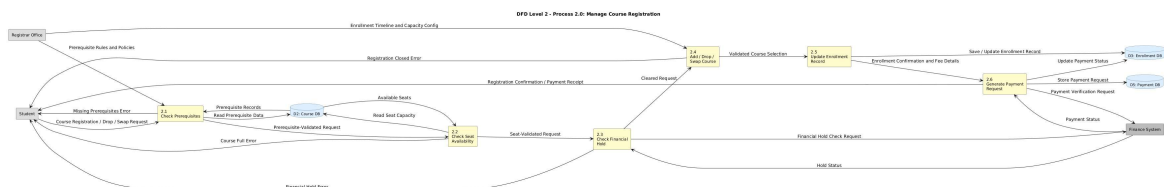


Figure 11: DFD Level 2 — Process 2.0: Manage Course Registration

Process 3.0 — Manage Courses and Grades

- **3.1 Create / Update Course:** Instructors and the Registrar can create or update course records in the Course DB.
- **3.2 Manage Course Roster:** Reads enrolled students from the Enrollment DB and returns the current roster to the instructor.
- **3.3 Upload Course Materials:** Stores a reference to uploaded materials in the Course DB.
- **3.4 Submit Grades:** Receives grade data from the instructor and reads the enrolled student list to validate submissions.
- **3.5 Validate and Store Grades:** Validates the submitted grade data and writes it to the Grades DB. Confirms back to the instructor.
- **3.6 Post Announcement:** Stores instructor announcements in the Course DB so enrolled students can access them.

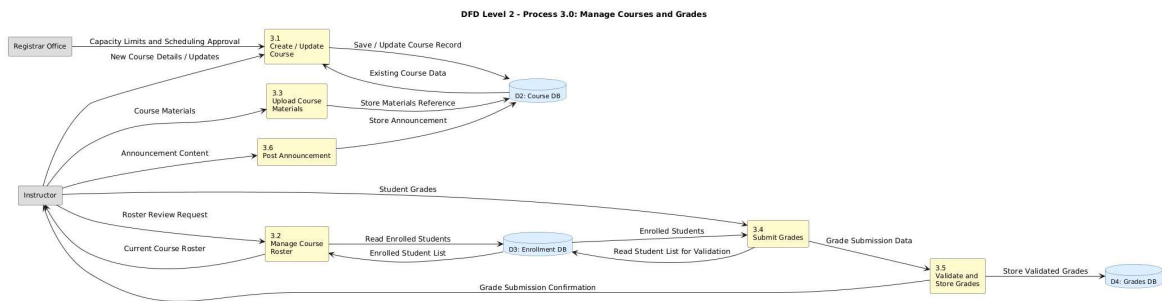


Figure 12: DFD Level 2 — Process 3.0: Manage Courses and Grades

Process 4.0 — Process Payments

- **4.1 Calculate Registration Fees:** Reads fee data from the Course DB and computes the total amount owed.
- **4.2 Send Payment Request:** Forwards the fee amount to the Finance System and shows the student a payment prompt.
- **4.3 Verify Payment:** Receives the payment status back from the Finance System.
- **4.4 Store Payment Record:** Writes the verified transaction to the Payment DB.
- **4.5 Update Enrollment Status:** Updates the Enrollment DB to reflect payment completion and sends the student a confirmation receipt.

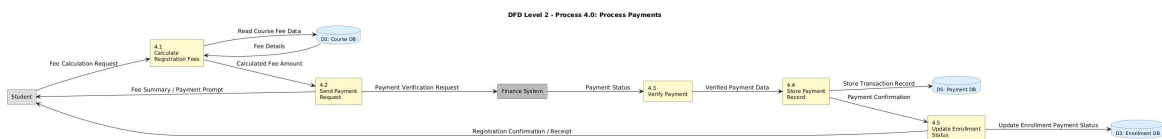


Figure 13: DFD Level 2 — Process 4.0: Process Payments

Process 5.0 — View Academic Records

- **5.1 Retrieve Student Profile:** Looks up the student's profile in the User DB using their ID.
- **5.2 Fetch Enrollment History:** Queries the Enrollment DB for all courses and semesters associated with that student.
- **5.3 Fetch Grade Records:** Uses the enrollment data to pull matching grades from the Grades DB.
- **5.4 Build Academic History:** Combines enrollment and grade data into a single compiled academic history.
- **5.5 Generate and Export Transcript:** Formats the compiled history and sends it to the external Transcript System for official generation, while also displaying the schedule, grades, and transcript view to the student.

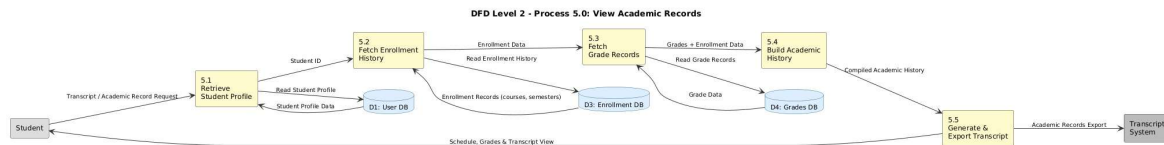


Figure 14: DFD Level 2 — Process 5.0: View Academic Records